Postconditions

The number of locks on `*this` held by the calling thread is reduced by one.

Throws

Nothing.

D.5.5. std::lock_guard class template

The `std::lock_guard` class template provides a basic lock ownership wrapper. The type of mutex being locked is specified by template parameter `Mutex` and must meet the `Lockable` requirements. The specified mutex is locked in the constructor and unlocked in the destructor. This provides a simple means of locking a mutex for a block of code and ensuring that the mutex is unlocked when the block is left, whether that's by running off the end, by the use of a control flow statement such as `break` or `return`, or by throwing an exception.

Instances of `std::lock_guard` are not `MoveConstructible`, `CopyConstructible`, or `CopyAssignable`.

Class definition

```
template <class Mutex>
class lock_guard
{
public:
    typedef Mutex mutex_type;

    explicit lock_guard(mutex_type& m);
    lock_guard(mutex_type& m, adopt_lock_t);
    ~lock_guard();

    lock_guard(lock_guard const& ) = delete;
    lock_guard& operator=(lock_guard const& ) = delete;
};
```

Std::Lock_Guard Locking Constructor